

Jupyter Notebook 원격 커널 구성 가이드

노트북 코드를 102 번이 아닌 101 번(cl-sdk) 서버에서 실행하기

항목	192.168.1.101	192.168.1.102
역할	커널 실행 노드 (코드가 실제로 도는 곳)	웹 화면 · 노트북 저장 (프론트엔드)
설치	cl-sdk + Jupyter Kernel Gateway	Jupyter Notebook (본 가이드 4 부에서 설치)
포트	8888 (Kernel Gateway)	8888 (웹 UI)
핵심	systemd 자동실행 + 토큰 + 방화벽	GatewayClient.url → 101 로 연결

이 문서 버전

대상 운영체제: Ubuntu Server (Resolute Raccoon) 26.04 LTS

아키텍처(CPU 종류): arm64 (ARM 64 비트)

전 과정을 root 계정으로 진행합니다.

0. 준비 — 서버 가상화 환경 만들기 (가장 먼저!)

이 가이드는 실제 서버 장비 2 대가 없어도 됩니다. 내 컴퓨터 한 대 안에 ‘가상 서버’ 2 대를 만들어 실습합니다. 순서는 간단합니다: ① 가상화 소프트웨어를 설치하고 → ② 그 위에 리눅스(운영체제)를 설치해 → ③ 가상 서버 2 대(101 번·102 번)를 만든 뒤 → ④ 본문(1 부~)을 따라 합니다.

0-1. 가상화가 무엇인가요? (1 분 이해)

- 가상화 소프트웨어(하이퍼바이저)는 내 컴퓨터 안에 독립된 ‘가상 컴퓨터(VM, Virtual Machine)’를 만들어 줍니다.
- 이 VM 안에 리눅스를 설치하면 별도 장비 없이 진짜 서버처럼 사용할 수 있습니다.
- 이 가이드는 VM 2 대가 필요합니다 — **101 번**(cl-sdk 로 코드 실행)과 **102 번**(Jupyter 웹 화면).

0-2. 내 컴퓨터 종류 확인 (아주 중요!)

이 문서는 arm64(ARM) 전용입니다

진행 가능한 컴퓨터: Apple Silicon Mac(M1·M2·M3·M4) 또는 ARM 기반 Windows PC(Snapdragon 등).

Intel/AMD CPU 의 일반 PC 에서는 이 무료 도구들로 arm64 리눅스를 설치할 수 없습니다 → 그 경우 ‘x64(x86_64)’ 버전 문서를 사용하세요.

CPU 종류 확인 방법 — **Windows**: 설정 → 시스템 → 정보 → ‘시스템 종류’ / **Mac**: 화면 왼쪽 위 사과(🍏) → ‘이 Mac 에 관하여’ → ‘칩’ 또는 ‘프로세서’ 항목.

아래 표는 어떤 컴퓨터(호스트)에서 어떤 버전(게스트)을 설치할 수 있는지 정리한 것입니다.

내 컴퓨터(호스트)	x64(x86_64) 설치	arm64(aarch64) 설치	사용 도구
Intel/AMD Windows PC	가능	불가	VMware Workstation Pro · VirtualBox
Intel Mac	가능	불가	VMware Fusion · VirtualBox
Apple Silicon Mac (M1~M4)	불가	가능	VMware Fusion · VirtualBox
ARM Windows PC (Snapdragon)	불가	가능	VirtualBox (7.2 이상)

→ 이 Ubuntu 26.04 LTS arm64 문서는 위 표에서 ‘**arm64 설치 = 가능**’ 인 컴퓨터에서 진행하세요.

0-3. 가상화 소프트웨어 다운로드 및 설치

아래 둘 중 하나만 설치하면 됩니다. 처음이라면 무료이고 설치가 쉬운 VirtualBox 도 좋은 선택입니다. (VMware 는 더 강력하지만 무료 Broadcom 계정 가입이 필요합니다.)

VMware 는 이제 개인·상업용 모두 무료입니다

2024 년 11 월부터 VMware Workstation Pro(Windows/Linux 용)와 VMware Fusion(Mac 용)이 무료가 되었습니다. 라이선스 키는 필요 없습니다.

다만 다운로드하려면 무료 Broadcom 계정으로 로그인해야 합니다(익명 다운로드 불가). 설치 중 ‘License Key’ 화면에서 ‘Personal Use(개인용)’를 선택하면 됩니다.

① Windows 에서 (Intel/AMD PC)

- VMware Workstation Pro(최신, 무료) — Broadcom 계정 가입:
<https://profile.broadcom.com/web/registration>
- VMware 다운로드 방법 공식 안내(로그인 후 다운로드): [Broadcom 지원 포털 안내 문서](#)
- VirtualBox(최신 7.2.x, 무료) — 공식 다운로드: <https://www.virtualbox.org/wiki/Downloads> → ‘Windows hosts’ 선택

② Mac 에서

- VMware Fusion(최신, 무료, Intel·Apple Silicon 모두 지원) — Broadcom 계정 가입:
<https://profile.broadcom.com/web/registration>
- VMware 다운로드 방법 공식 안내(로그인 후 다운로드): [Broadcom 지원 포털 안내 문서](#)
- VirtualBox(최신 7.2.x, 무료) — 공식 다운로드: <https://www.virtualbox.org/wiki/Downloads> → Intel Mac 은 ‘macOS / Intel hosts’, Apple Silicon 은 ‘macOS / Apple Silicon hosts’ 선택

arm64 사용자 주의

Apple Silicon Mac 이라면 VMware Fusion 또는 VirtualBox(7.2 이상) 중 하나를 쓰면 됩니다.

ARM 기반 Windows PC 라면 VirtualBox(7.2 이상)만 사용 가능합니다(VMware Workstation Pro 는 Intel/AMD 전용).

0-4. 운영체제(OS) 설치 이미지(ISO) 다운로드

이 문서는 **Ubuntu Server (Resolute Raccoon) 26.04 LTS (arm64)** 용입니다. 아래에서 해당 ISO 파일 1 개를 내려받으세요. 가장 무난한 것은 ‘DVD’ 이미지입니다.

항목	내용
배포판/버전	Ubuntu Server (Resolute Raccoon) 26.04 LTS
아키텍처	arm64 (ARM 64 비트)
내려받을 파일	ubuntu-26.04-live-server-arm64.iso (arm64 · Server)
공식 다운로드	ubuntu.com/download/server/arm

다운로드 참고

Ubuntu 는 계정 없이 바로 내려받을 수 있습니다. ‘Ubuntu Server for ARM’ 페이지에서 26.04 LTS arm64 ISO 를 받으세요(또는 cdimage.ubuntu.com/releases/26.04/). 데스크톱 버전은 필요 없습니다.

0-5. 가상 서버(VM) 2 대 만들기 + 리눅스 설치

VMware/VirtualBox 공통 순서입니다. ‘새 가상 머신 만들기’ → 위에서 받은 ISO 지정 → 사양(메모리·디스크) 설정 → 커서 설치 진행(언어·시간대 선택, 사용자/비밀번호 만들기). 이 작업을 2 번 반복해 101 번·102 번을 만듭니다.

권장 사양

가상 서버	역할	CPU	메모리(RAM)	디스크
101 번	cl-sdk + Kernel Gateway	2 개 이상	4GB 이상	30GB 이상
102 번	Jupyter 웹 화면	2 개	2GB 이상	20GB 이상

네트워크 모드는 ‘브리지(Bridged)’ 권장

두 가상 서버가 서로 통신해야 하므로, 각 VM 의 네트워크 설정을 ‘브리지(Bridged Adapter)’로 두면 가장 쉽습니다. 그러면 두 서버가 같은 공유기 네트워크(192.168.1.x)에 붙어 서로 통신할 수 있습니다.

공유기 환경에 따라 192.168.1.101 / 192.168.1.102 주소가 비어 있는지(다른 기기가 안 쓰는지) 확인하세요. 사용 중이라면 라우터 DHCP 할당 범위 밖의 빈 주소로 바꾸고, 본문 IP 도 동일하게 맞추면 됩니다.

VMware 에서 ‘브리지’ 대신 같은 ‘NAT 네트워크’ 에 두어도 두 VM 은 서로 통신할 수 있습니다(선택).

arm64 설치 화면 참고

Apple Silicon Mac + VMware Fusion 에서 새 VM 을 만들 때는 ‘Create a custom virtual machine → Linux → 해당 배포판(또는 기타 Linux 6.x kernel 64-bit Arm)’ 식으로 선택하고, 위에서 받은 arm64 ISO 를 지정하면 됩니다.

0-6. 두 서버에 고정 IP 설정

본문은 고정 IP 를 사용합니다. **101 번 = 192.168.1.101, 102 번 = 192.168.1.102** 로 각각 설정하세요. 아래 예시는 101 번 기준이며, 102 번에서는 주소만 .102 로 바꿔 동일하게 진행합니다.

Ubuntu Server 는 netplan 으로 설정합니다. 아래 명령을 그대로 실행하면 설정 파일이 만들어집니다.

```
ip -br link                                # 네트워크 장치 이름 확인 (예: eth0, ens33, enp0s3)
# 아래 'eth0'은 위에서 확인한 실제 장치 이름으로 바꾸세요.
cat > /etc/netplan/99-static.yaml << 'EOF'
network:
  version: 2
  ethernets:
    eth0:
      dhcp4: false
      addresses: [192.168.1.101/24]
      routes:
        - to: default
          via: 192.168.1.1
      nameservers:
        addresses: [8.8.8.8, 1.1.1.1]
EOF
chmod 600 /etc/netplan/99-static.yaml
netplan apply
```

Ubuntu 주의

YAML 은 들여쓰기에 민감합니다. 위 heredoc(cat ... EOF) 방식으로 만들면 들여쓰기가 그대로 유지됩니다. 직접 편집할 때는 공백 2 칸을 쓰고 탭(Tab)은 쓰지 마세요.

102 번에서는 addresses 를 [192.168.1.102/24] 로 바꿉니다.

설정 후 두 서버가 서로 통신되는지 확인합니다(102 번에서 실행). 아래가 응답하면 준비 끝 — **이제 1 부부터 진행합니다.**

```
ping -c 2 192.168.1.101
```

Ubuntu 는 root 로 전환 후 진행

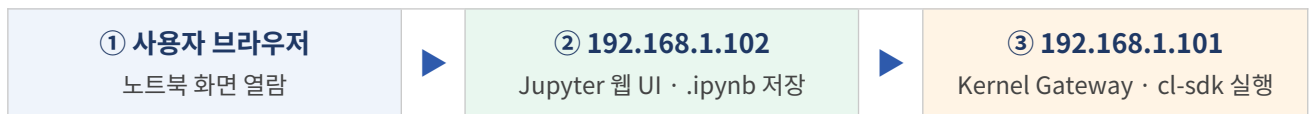
Ubuntu 는 기본적으로 root 로그인에 막혀 있습니다. 설치 때 만든 일반 계정으로 로그인한 뒤, 아래 명령으로 root 로 전환해 이후 모든 작업을 진행하세요.

```
sudo -i
```

1. 개요

이 문서는 102 번의 Jupyter 웹 화면에서 코드를 실행할 때, 실제 파이썬 커널이 cl-sdk 가 설치된 101 번 서버에서 동작하도록 구성하는 방법을 설명합니다.

Jupyter 는 ‘화면(노트북 편집·저장)’과 ‘커널(코드 실행)’이 분리되어 있습니다. 101 번에 Kernel Gateway 를 두고 102 번 Jupyter 가 커널 관리를 101 번에 위임하면, 모든 코드 실행이 101 번에서 이루어집니다. 노트북 편집과 저장은 그대로 102 번에서 합니다.



위 구조에서 노트북 셀을 실행하면 → 102 가 101 의 Kernel Gateway 에 커널 생성을 요청 → 커널은 101 에서 기동 → `import cl` 등 모든 코드가 101 에서 실행되고 결과만 102 화면으로 돌아옵니다.

2. 사전 점검 (양쪽 공통)

- 두 서버 모두 Ubuntu 26.04 LTS (arm64), root 계정으로 작업합니다.
- Ubuntu 는 일반 계정으로 로그인 후 `sudo -i` 로 root 로 전환한 상태에서 진행합니다.
- 101 번에 cl-sdk 가 설치되어 있고 `import cl` 이 정상 동작합니다. (1 차 설치 가이드 완료 상태)
- 101 번 Kernel Gateway(3 부)와 102 번 Jupyter Notebook(4 부)은 본 가이드에서 직접 설치합니다. 별도 사전 설치의 필요 없습니다.
- 101 ↔ 102 가 같은 네트워크 대역에서 통신 가능합니다. 102 번에서 아래로 확인합니다:

```
ping -c 2 192.168.1.101
```

3. [101 번] Kernel Gateway 설치 및 설정

단계 1. cl-sdk 가상환경에 Kernel Gateway 추가 설치

경로 안내

아래는 venv 경로를 `/opt/cl-sdk/venv` 로 가정합니다. 이미 cl-sdk 를 설치한 venv 가 있으면 그 경로로 바꿔서 진행하세요.

```
mkdir -p /opt/cl-sdk && cd /opt/cl-sdk
test -d venv || python3 -m venv venv
source venv/bin/activate
pip install --upgrade pip
```

```
pip install cl-sdk jupyter_kernel_gateway
```

cl-sdk는 ipykernel을 포함하므로, 같은 venv에 Kernel Gateway만 추가하면 'Python 3' 커널에서 곧바로 import cl을 사용할 수 있습니다. pip가 현재 아키텍처(arm64)에 맞는 aarch64(arm64)용 wheel을 자동으로 설치합니다.

단계 2. 공유 인증 토큰 생성 (101과 102가 같은 값 사용)

```
openssl rand -hex 32
```

출력된 문자열을 복사해 두세요. 이 **동일한 토큰**을 101번 서비스 파일과 102번 설정 파일 양쪽에 똑같이 입력합니다. (아래 예시의 PUT-YOUR-SHARED-TOKEN-HERE 자리)

단계 3. systemd 서비스 등록 (재부팅 시 자동 실행)

아래 내용으로 서비스 파일을 만듭니다. 토큰 값과 venv 경로를 환경에 맞게 바꾸세요.

```
cat > /etc/systemd/system/cl-kernel-gateway.service << 'EOF'
[Unit]
Description=Jupyter Kernel Gateway (cl-sdk)
After=network-online.target
Wants=network-online.target

[Service]
Type=simple
User=root
Environment=KG_AUTH_TOKEN=PUT-YOUR-SHARED-TOKEN-HERE
ExecStart=/opt/cl-sdk/venv/bin/jupyter kernelgateway \
  --KernelGatewayApp.ip=0.0.0.0 \
  --KernelGatewayApp.port=8888 \
  --KernelGatewayApp.api=kernel_gateway.jupyter_websocket
Restart=on-failure
RestartSec=3

[Install]
WantedBy=multi-user.target
EOF
```

서비스를 활성화하고 부팅 시 자동 실행되도록 등록합니다:

```
systemctl daemon-reload
systemctl enable --now cl-kernel-gateway
systemctl status cl-kernel-gateway --no-pager
```

단계 4. 방화벽 허용 — 102번만 접근 허용 (권장)

```
ufw allow OpenSSH # SSH로 원격 접속 중이면 먼저 허용(접속 끊김 방지)
ufw allow from 192.168.1.102 to any port 8888 proto tcp
ufw --force enable
ufw reload
```

단계 5. 게이트웨이 동작 확인 (101 번에서 실행)

토큰을 넣어 커널 목록을 조회합니다. python3 커널이 보이면 정상입니다.

```
curl -s -H "Authorization: token PUT-YOUR-SHARED-TOKEN-HERE" \
http://192.168.1.101:8888/api/kernelspecs
```

4. [102 번] Jupyter Notebook 설치

102 번은 웹 화면(프론트엔드) 역할만 하므로 cl-sdk 는 필요 없고 Jupyter Notebook 만 설치합니다. 코드는 101 번 커널에서 실행되므로 102 번에는 별도 커널이 없어도 됩니다.

단계 1. Python 3 / pip 확인

기본으로 Python 3 가 포함되어 있습니다(버전은 아래 명령으로 확인). 가상환경(venv) 생성을 위해 python3-venv 패키지를 함께 설치합니다.

```
python3 --version
apt update
apt install -y python3 python3-pip python3-venv
```

단계 2. 전용 가상환경 생성 후 Notebook 설치

```
mkdir -p /opt/jupyter && cd /opt/jupyter
test -d venv || python3 -m venv venv
source venv/bin/activate
pip install --upgrade pip
pip install notebook
```

Notebook 7.x 가 설치됩니다. arm64 용 wheel 이 제공되어 별도 빌드 도구 없이 설치됩니다.

단계 3. 기본 설정 파일 작성

root 계정의 설정 파일에 접속 주소·포트·root 실행 허용을 지정합니다. 0.0.0.0 으로 바인딩하면 외부 브라우저에서 접속할 수 있습니다.

```
mkdir -p /root/.jupyter /root/notebooks
cat > /root/.jupyter/jupyter_server_config.py << 'EOF'
c.ServerApp.ip = "0.0.0.0"
c.ServerApp.port = 8888
c.ServerApp.open_browser = False
```

```
c.ServerApp.allow_root = True
c.ServerApp.allow_remote_access = True
c.ServerApp.root_dir = "/root/notebooks"
EOF
```

참고

root 로 실행하므로 c.ServerApp.allow_root = True 가 필요합니다. 이 설정이 없으면 Jupyter 가 root 실행을 거부합니다.

단계 4. 접속 비밀번호 설정

브라우저 접속용 비밀번호를 설정합니다. 토큰 대신 비밀번호 방식이 운영에 편리합니다.

```
/opt/jupyter/venv/bin/jupyter server password
```

비밀번호를 두 번 입력하면 해시가 /root/.jupyter/jupyter_server_config.json 에 안전하게 저장됩니다.

단계 5. systemd 서비스 등록 (재부팅 시 자동 실행)

아래 내용으로 서비스 파일을 만듭니다. venv 경로를 환경에 맞게 바꾸세요.

```
cat > /etc/systemd/system/jupyter.service << 'EOF'
[Unit]
Description=Jupyter Notebook
After=network-online.target
Wants=network-online.target

[Service]
Type=simple
User=root
ExecStart=/opt/jupyter/venv/bin/jupyter notebook \
  --config=/root/.jupyter/jupyter_server_config.py
Restart=on-failure
RestartSec=3

[Install]
WantedBy=multi-user.target
EOF
```

```
systemctl daemon-reload
systemctl enable --now jupyter
systemctl status jupyter --no-pager
```

단계 6. 방화벽 허용 및 접속 확인

웹 접속 포트 8888 을 엽니다. 필요 시 접속 출발지를 제한하세요.


```
ufw allow OpenSSH          # SSH로 원격 접속 중이면 먼저 허용(접속 끊김 방지)
ufw allow 8888/tcp
ufw --force enable
ufw reload
```

이제 브라우저에서 **http://192.168.1.102:8888** 로 접속해 설정한 비밀번호로 로그인합니다.

다음 단계

여기까지가 Jupyter 설치입니다. 이어서 5 부에서 이 Jupyter가 101 번 커널을 사용하도록 게이트웨이 연결을 추가합니다.

5. [102 번] Gateway Client 설정

단계 1. Jupyter 설정 파일에 게이트웨이 연결 추가

root 계정의 Jupyter 서버 설정 파일에 아래 4 줄을 추가합니다. 토큰은 **101 번과 동일한 값**이어야 합니다.

```
mkdir -p /root/.jupyter
cat >> /root/.jupyter/jupyter_server_config.py << 'EOF'
c.GatewayClient.url = "http://192.168.1.101:8888"
c.GatewayClient.auth_token = "PUT-YOUR-SHARED-TOKEN-HERE"
c.GatewayClient.connect_timeout = 60.0
c.GatewayClient.request_timeout = 60.0
EOF
```

동작 핵심

이 설정이 적용되면 102 번 Jupyter 는 커널 생성·목록을 모두 101 번에 위임합니다. 즉 화면은 102 번, 코드 실행은 101 번 이 됩니다.

단계 2. Jupyter 서비스 재시작 (게이트웨이 설정 반영)

4 부에서 등록한 Jupyter 서비스를 재시작하면 위 게이트웨이 설정이 반영됩니다.

```
systemctl restart jupyter
systemctl status jupyter --no-pager
```

6. 동작 검증

102 번 브라우저에서 Jupyter 에 접속해 **새 노트북(Python 3)**을 열고 아래 셀을 차례로 실행합니다. 결과가 101 번을 가리키면 성공입니다.

① 커널이 실행되는 호스트 확인

```
import socket
print("커널 실행 호스트:", socket.gethostname())
```

→ 101 번 서버의 호스트네임이 출력되어야 합니다. (101 번에서 hostname 으로 확인한 값과 동일)

② IP 확인

```
import subprocess
print(subprocess.run(["hostname", "-I"], capture_output=True, text=True).stdout.strip())
```

→ 출력에 192.168.1.101 이 포함되어야 합니다.

③ cl-sdk 로드 확인

```
import cl
print("cl-sdk 로드 성공:", cl.__name__)
```

7. 문제 해결

증상	원인	해결
커널이 'Unknown' / 연결 안 됨	101·102 토큰 불일치	양쪽 토큰을 동일하게 맞추고 102 서비스 재시작
실행 시 멈춤 / 연결 실패	방화벽 차단	101 에서 102 출발지만 허용(ufw)
403 / Forbidden	인증 토큰 누락	102 설정의 auth_token 값 확인
import cl 실패 (ModuleNotFound)	게이트웨이가 cl-sdk 없는 파이썬으로 기동	101 서비스 ExecStart 가 cl-sdk venv 의 jupyter 인지 확인
재부팅 후 동작 안 함	서비스 enable 누락	101: systemctl enable --now cl-kernel-gateway
커널 목록이 비어 있음	101 게이트웨이 미기동	101: systemctl status cl-kernel-gateway 로 상태 점검
102 웹 접속 불가 (102:8888)	방화벽 차단 / 바인딩 오류	102: ufw allow 8888/tcp 후 ufw reload, c.ServerApp.ip='0.0.0.0' 확인
root 실행 거부	allow_root 미설정	102 설정에 c.ServerApp.allow_root=True 추가 후 재시작

8. 보안 및 데이터 보호 메모

- 토큰은 추측 불가능한 값(openssl rand -hex 32)을 사용하고, 설정/서비스 파일을 root 전용으로 제한합니다.
- 방화벽으로 8888 포트를 102 번에서만 접근하도록 제한합니다. (3 부 단계 4)

- 본 구성은 전 구간이 **사내 폐쇄망(101·102) 내부**에서만 통신하며, 코드·데이터가 외부(인터넷·제 3 자)로 전송되지 않습니다.
- 외부망 노출이 필요하면 평문(HTTP) 대신 TLS(인증서)와 추가 접근 통제를 적용하십시오.

파일 권한 제한:

```
chmod 600 /etc/systemd/system/cl-kernel-gateway.service # 101
chmod 600 /root/.jupyter/jupyter_server_config.py # 102 (게이트웨이 토큰 포함)
chmod 600 /root/.jupyter/jupyter_server_config.json # 102 (비밀번호 해시)
```

부록. 전체 명령 요약

A. 101 번 서버

```
sudo -i # root로 전환
# 1) cl-sdk venv + Kernel Gateway
mkdir -p /opt/cl-sdk && cd /opt/cl-sdk
test -d venv || python3 -m venv venv
source venv/bin/activate
pip install --upgrade pip
pip install cl-sdk jupyter_kernel_gateway

# 2) 공유 토큰 생성 (출력값을 양쪽에 동일 입력)
openssl rand -hex 32

# 3) systemd 서비스 등록 후 자동 실행
# (위 본문 3부 단계 3의 서비스 파일 생성 후)
systemctl daemon-reload
systemctl enable --now cl-kernel-gateway

# 4) 방화벽: 102 번만 8888 허용
ufw allow OpenSSH # SSH로 원격 접속 중이면 먼저 허용(접속 끊김 방지)
ufw allow from 192.168.1.102 to any port 8888 proto tcp
ufw --force enable
ufw reload
```

B. 102 번 서버

```
sudo -i # root로 전환
# 1) Jupyter Notebook 설치 (전용 venv)
apt update && apt install -y python3 python3-pip python3-venv
mkdir -p /opt/jupyter && cd /opt/jupyter
test -d venv || python3 -m venv venv
source venv/bin/activate
pip install --upgrade pip
pip install notebook
```

```
# 2) 기본 설정 파일 작성
mkdir -p /root/.jupyter /root/notebooks
cat > /root/.jupyter/jupyter_server_config.py << 'EOF'
c.ServerApp.ip = "0.0.0.0"
c.ServerApp.port = 8888
c.ServerApp.open_browser = False
c.ServerApp.allow_root = True
c.ServerApp.allow_remote_access = True
c.ServerApp.root_dir = "/root/notebooks"
EOF

# 3) 접속 비밀번호 설정 (대화형)
/opt/jupyter/venv/bin/jupyter server password

# 4) 게이트웨이 연결 추가 (토큰은 101 과 동일)
cat >> /root/.jupyter/jupyter_server_config.py << 'EOF'
c.GatewayClient.url = "http://192.168.1.101:8888"
c.GatewayClient.auth_token = "PUT-YOUR-SHARED-TOKEN-HERE"
c.GatewayClient.connect_timeout = 60.0
c.GatewayClient.request_timeout = 60.0
EOF

# 5) systemd 서비스 등록 후 자동 실행 (서비스 파일은 본문 4부 단계 5 참고)
systemctl daemon-reload
systemctl enable --now jupyter

# 6) 방화벽: 웹 접속 8888 허용
ufw allow OpenSSH                # SSH로 원격 접속 중이면 먼저 허용(접속 끊김 방지)
ufw allow 8888/tcp
ufw --force enable
ufw reload
```